Lab-1

Implement and demonstrate the find s algorithm for finding the most specific hypothesis. based on given set of training data samples. Read the data from .csv file.

Dataset:

Outlook	Temp	Humidity	Windy	Play Tennis
Sunny	Hot	high	false	Yes
Sunny	Hot	high	false	Yes
Rainy	cool	high	false	No
Sunny	cool	high	false	Yes
Sunny	mild	high	false	Yes

```
import pandas as pd
def find_s_algorithm(first):
    features = first.iloc[:, :-1].values
    labels = first.iloc[:, -1].values
    hypothesis = features[labels == 'Yes'][0]
    for i in range(1, len(features)):
        if labels[i] == 'Yes':
            for j in range(len(hypothesis)):
                if hypothesis[j] != features[i][j]:
                    hypothesis[j] = '?'
    return hypothesis
first = pd.read_csv("C:/Users/student/Desktop/p1.csv")
hypothesis = find_s_algorithm(first)
print('The most specific hypothesis is :", hypothesis)
```

To read a file :

```
with open("c:/users/student/Desktop/ml.text", 'r') as file:
    content = file.read()
print(content)
```

O/p: Machine learning
System compilers
Blockchain technology.

To write into a file :

```
with open("c:/users/student/Desktop/ml.txt", 'w') as file:
    content = "writing from python file"
    file.write(content)
print("Written successfully into file")
```

O/p : written successfully into file.

O/p :

The most specific hypothesis is : ['Sunny', '?', 'High', '?']

12/2

Lab-2

2) Write a program to implement the naive Bayesian classifier for a simple training dataset stored as a .csv file. Compute the accuracy of the classifier, considering few test datasets.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
filename = "data.csv"
df = pd.read_csv(filename, delimiter = ';')
→ df.fillna(df.mean(), inplace = True)
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
  0.2, random_state = 42)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Naive Bayes classifier accuracy: {accuracy * 100:.2f}%")
```

Dataset :

Age	Salary	Loan	Default
25	50000	No	No
30	60000	Yes	Yes
22	40000	No	No
35	80000	Yes	Yes

O/p : Naive Bayes classifier accuracy : 55.00%

19/3/25

Lab - 3

3. Build an artificial neural network by implementing the backpropagation algorithm and test the same using appropriate datasets

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1-x)
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size,
        learning rate = 0.5):
            self.input_size = input_size
            self.hidden_size = hidden_size
            self.output_size = output_size
            self.learning_rate = learning_rate
            self.weights_input_hidden = np.random.uniform(-1, 1,
            (self.input_size, self.hidden_size))
            self.weights_hidden_output = np.random.uniform(-1, 1,
            (self.hidden_size, self.output_size))
            self.bias_hidden = np.random.uniform(-1, 1, (1, self.
              hidden_size))
            self.bias_output = np.random.uniform(-1, 1, (1, self.
             output_size))
```

```
def forward(self, X):
    self.hidden_input = np.dot(X, self.weights_inputs_hidden) +
    self.bias_hidden
    self.hidden_output = sigmoid(self.hidden_input)
    self.final_input = np.dot(self.hidden_output, self.weights_
    hidden_output) + self.bias_output
    self.final_output = sigmoid(self.final_input)
    return self.final_output

def backward(self, X, y, output):
    output_error = y - output
    output_delta = output_error * sigmoid_derivative(output)
    hidden_error = output_delta.dot(self.weights_hidden_output.T)
    hidden_delta = hidden_error * sigmoid_derivative(self.hidden_
                                                     output)
    self.weights_hidden_output += self.hidden_output.T.dot(output_
    delta) * self.learning_rate
    self.bais_output += np.sum(output_delta, axis=0, keepdims=True)
    self.learning_rate
    self.weights_input_hidden += X.T.dot(hidden_delta) *
    self.learning_rate
    self.bias_hidden += np.sum(hidden_delta, axis=0, keepdims=
    True) * self.learning_rate

def train(self, X, y, epochs=10000):
    for epoch in range(epochs):
        output = self.forward(X)
```

```
        self.backward(X, y, output)
        if epoch % 1000 == 0:
            loss = np.mean(np.square(y - output))
            print(f"Epoch {epoch}, Loss: {loss}")

    def predict(self, X):
        return self.forward(X)

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [1], [1], [0]])
nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1, learning_rate=0.5)
nn.train(X, y, epochs=10000)
predictions = nn.predict(X)
print("Predictions:")
print(predictions)
```

O/P:

```
Epoch 0, Loss = 0.2619
Epoch 1000, Loss = 0.0072
Epoch 2000, Loss = 0.0017
Epoch 3000, Loss = 0.0009
Epoch 4000, Loss = 0.0006
Epoch 5000, Loss = 0.0004
Epoch 6000, Loss = 0.0003
Epoch 7000, Loss = 0.0003
Epoch 8000, Loss = 0.0002
Epoch 9000, Loss = 0.0002
```

Predictions:

```
[[0.01233]
 [0.98486]
 [0.98551]
 [0.01549]]
```

1113